Architectural Foundations of Tabular Machine Learning: High-Performance Pure-Function Implementations for Gradient Boosting Pipelines

The architectural design of robust machine learning systems mandates a strict decoupling between stateful model training and stateless data transformation. In tabular machine learning—a domain frequently dominated by gradient boosted decision trees (GBDT) such as LightGBM, XGBoost, and CatBoost—preprocessing pipelines often suffer from significant technical debt. This debt typically manifests as memory bloat, high execution latency, and side-effect mutations introduced by high-level data manipulation libraries like Pandas. To achieve best-in-class performance, continuous integration readiness, and absolute determinism, it is necessary to distill these complex pipelines into "ML Atoms": pure, stateless functions implemented over contiguous memory buffers utilizing NumPy and SciPy.

This comprehensive report systematically investigates optimal pure-function implementations for tabular machine learning preprocessing, advanced feature engineering, and GBDT post-processing. The research focuses on identifying high-performance, memory-efficient, and vectorized formulations that satisfy stringent software contracts, explicitly targeting integration into the sciona-atoms-ml repository. By treating the core gradient boosting model training as an opaque, compiled C++ atom, this analysis spans the critical peripheral stages: data assembly, temporal signal transformation, structural feature engineering, and analytical loss decompositions.

The Paradigm of Pure-Function Machine Learning Atoms

Before delving into specific implementations, it is crucial to establish the foundational philosophy guiding this research. A pure function, in the context of mathematical and computer science literature, is a function where the return value is determined strictly by its input values, without observable side effects. In the ecosystem of data science, tools like Pandas and Scikit-Learn frequently violate this paradigm. DataFrames carry complex metadata indices that can be inadvertently mutated, and stateful transformers (e.g., StandardScaler) conflate the estimation of parameters with the transformation of the data array.

The sciona-atoms-ml repository seeks to eradicate these inefficiencies by defining transformations strictly as functions operating on numpy.ndarray objects. This approach forces the separation of the "fit" stage (which outputs a deterministic dictionary or array of parameters) from the "transform" stage (which applies those parameters without altering them). Furthermore,

minimizing dependencies to pure NumPy or SciPy ensures that the computational graphs can be efficiently JIT-compiled via Numba or ported to GPU accelerators via CuPy, bypassing the Global Interpreter Lock (GIL) limitations inherent to Python object manipulation.[2]

Part I: Data Assembly and Aggregation Atoms

Relational tabular data, such as the schemas encountered in the Home Credit Default Risk and Instacart Market Basket Analysis challenges, necessitates joining, aggregating, and summarizing high-cardinality child tables into flattened parent entity representations. Performing these operations without Pandas requires low-level array manipulation and an intimate understanding of stride mechanics.

Entity Aggregation and Group-By Statistics

The split-apply-combine paradigm is the cornerstone of relational data analysis.[3] While Pandas utilizes heavily optimized Cython routines for these operations, pure NumPy implementations can achieve comparable or even superior performance by minimizing object-oriented overhead and leveraging C-level array loops.[4]

When analyzing pure NumPy implementations, several algorithmic approaches present themselves. The masking approach, which utilizes boolean arrays (e.g., values[keys == key].sum()), is highly intuitive but scales abysmally. Its computational complexity is $O(N \cdot M)$, where N is the number of rows and M is the number of unique groups, due to the necessity of full-array conditional passes for every single group.[3]

The optimal vectorized approach for summation and mean calculations relies on numpy.bincount, which computes weighted sums in a single contiguous pass, yielding an $O(N)$ time complexity.[3] Because numpy.bincount requires a dense array of non-negative integers as bins, arbitrary group identifiers (such as strings or sparse UUIDs) must first be mapped to a contiguous integer space. This is achieved using numpy.unique with the return_inverse=True parameter, which efficiently broadcasts the original keys into an array of positional indices suitable for binning.[3]

For non-summation metrics—such as minimum, maximum, first, and last—the bincount methodology is mathematically insufficient. Advanced implementations must rely on sorting the array by group keys via numpy.argsort, calculating the boundaries of each group using numpy.diff, and applying specialized reduction functions like numpy.maximum.reduceat over the sorted contiguous blocks.[7]

The following table summarizes the performance characteristics of various group-by paradigms in Python:

Algorithmic Approach	Underlying Mechanism	Time Complexity	Suitability for sciona-atoms-ml
Pandas GroupBy	Cython-compiled hash maps	$O(N)$	Unsuitable (violates pure NumPy constraint, introduces metadata overhead).
NumPy Masking	Iterative boolean array generation	$O(N \cdot M)$	Unsuitable (computationally intractable for high-cardinality groups).
NumPy Bincount	Contiguous integer binning (np.bincount)	$O(N)$	Highly Suitable (optimal for sum, count, and mean operations).
NumPy Reduceat	Array sorting and boundary reduction	$O(N \log N)$	Highly Suitable (optimal for min, max, first, and last operations).
SciPy Sparse	Coordinate matrix multiplication	$O(N)$	Suitable, but introduces Scipy dependency overhead for simple tasks.

Libraries like numpy_groupies further optimize these operations using Just-In-Time (JIT) compilation via Numba, definitively proving that NumPy-native structures can bypass Pandas entirely while maintaining microsecond latency.[4]

Name: group_aggregate

Description: Compute entity-level group-by statistics using contiguous integer mapping and

weighted bin counting for summation metrics, and strided reductions for extremum metrics.

Source: https://jakevdp.github.io/blog/2017/03/22/group-by-from-scratch/

License: MIT

Concept type: data_assembly

Signature: (values: NDArray, groups: NDArray, agg_fn: str) -> NDArray

Pure function boundary: tabular arrays and explicit parameters in, 1D array of aggregated values out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(values) == len(groups)
- require: agg_fn in ['sum', 'mean', 'var', 'std', 'min', 'max', 'count', 'first', 'last']
- ensure: len(output) == len(np.unique(groups))
  Witness: random integer array grouped by a low-cardinality string array; verify results perfectly match an equivalent SQL GROUP BY query.
  Dependencies: numpy
  CDG stages covered: amex_default/entity_aggregation, instacart/user_statistics

Multi-Table Join and Aggregation

Aggregating a child table to a parent table involves computing the aforementioned group statistics and subsequently mapping those statistics back into the parent entity's dimensional space. This requires rigorously aligning the foreign keys of the child table with the primary keys of the parent table.[10]

The optimal pure-function NumPy implementation fundamentally avoids explicit Python iteration. Instead, it utilizes numpy.searchsorted or numpy.in1d to align the parent keys with the pre-aggregated child keys. If the child table has been reduced using the group_aggregate atom, the resulting key array is aligned with the parent key array to construct an appropriately shaped feature matrix. Non-matching parent keys (entities with no child records) are deterministically filled with a predefined fill_value, which is contextually either NaN or 0.[4] This operation guarantees that the resulting feature matrix perfectly aligns with the row order of the training data being fed into the gradient boosting model.

Name: aggregate_child_table

Description: Join pre-aggregated child table statistics onto a parent table using binary search alignment of primary and foreign keys.

Source: https://github.com/ml31415/numpy-groupies

License: MIT

Concept type: data_assembly

Signature: (child_values: NDArray, parent_keys: NDArray, child_keys: NDArray, agg_fns: list[str]) -> NDArray

Pure function boundary: tabular arrays and explicit parameters in, 2D array of aligned aggregates out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(child_values) == len(child_keys)
- require: len(parent_keys) == len(np.unique(parent_keys))
- ensure: output.shape == (len(parent_keys), len(agg_fns))
  Witness: large parent table and multiple child records; verify parent shape is maintained and child aggregates are correctly broadcasted with NaNs for missing keys.
  Dependencies: numpy
  CDG stages covered: home_credit/bureau_balance_aggregation, instacart/prior_orders_aggregation

Part II: Temporal Signal Transform Atoms

Temporal structures require highly specialized atoms to capture sequence dependencies, moving averages, and the degradation of signal relevance over time. These features are immensely prevalent in financial risk modeling, anomaly detection, and commercial forecasting.[13]

Difference Features

Computing the delta between consecutive time steps per entity captures the instantaneous rate of change and acceleration. In credit default prediction (e.g., the Amex Default Prediction competition), the temporal difference between the final statement observation and the penultimate observation provides critical momentum signals that heavily influence the tree splits.[15]

A pure NumPy implementation must gracefully handle variable-length sequences per entity. The raw input data is first lexicographically sorted by entity ID and timestamp using numpy.lexsort. Once strictly ordered, numpy.diff calculates the sequential differences across the entire column.[12] However, to prevent meaningless differences from being calculated across entity boundaries (e.g., subtracting Customer B's first observation from Customer A's last observation), a boolean mask is generated by applying numpy.diff to the entity IDs themselves. Differences where the entity ID changes are strictly masked out and replaced with NaN.[7]

Name: temporal_difference

Description: Compute the delta between consecutive time steps for specific features, explicitly

masking differences that span across disparate entity boundaries.

Source: https://numpy.org/doc/stable/reference/generated/numpy.diff.html

License: BSD-3-Clause

Concept type: signal_transform

Signature: (values: NDArray, entity_ids: NDArray, sort_keys: NDArray) -> NDArray

Pure function boundary: tabular arrays and explicit parameters in, contiguous difference arrays out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(values) == len(entity_ids) == len(sort_keys)
- ensure: output elements corresponding to the chronological first observation of any entity yield NaN.
  Witness: sequential sensor readings grouped by machine ID; verify the arithmetic difference is exact and boundary rows output NaN without exception.
  Dependencies: numpy
  CDG stages covered: amex_default/statement_momentum, home_credit/installment_differences

Rolling Window Statistics

Rolling statistics evaluated over entity-specific time windows serve to highlight localized volatility and trend stabilization while smoothing out high-frequency noise.[14] Standard Python iteration over sub-arrays is catastrophically slow for rolling operations, precluding its use in competitive machine learning.[18]

NumPy offers two highly optimized paradigms for resolving this computational bottleneck. The first paradigm utilizes numpy.cumsum to efficiently compute moving averages in absolute $O(N)$ time. This is achieved by subtracting the cumulative sum at index $t - \text{window}$ from the sum at index t, completely bypassing the need to sum the intermediate elements repeatedly.[19]

The second, substantially more versatile approach leverages deep memory strides. By deliberately altering the array's stride metadata (which dictates the number of bytes the pointer must jump to find the next element along a given axis), an array can be mathematically projected into overlapping windows without duplicating or copying the underlying data buffer.[20] This zero-copy matrix projection is formally implemented in NumPy version 1.20 and later via numpy.lib.stride_tricks.sliding_window_view.[21] By executing mathematical operations strictly

over the final axis of this newly generated strided view (e.g., np.std(windows, axis=-1)), rolling extremums, variances, and bounds are computed with maximum cache efficiency.[23]

Name: rolling_statistics

Description: Compute moving averages, minimums, maximums, and standard deviations utilizing zero-copy overlapping stride views over sorted entity arrays.

Source: https://numpy.org/devdocs/reference/generated/numpy.lib.stride_tricks.sliding_window_view.html

License: BSD-3-Clause

Concept type: signal_transform

Signature: (values: NDArray, window_size: int, agg_fns: list[str]) -> NDArray

Pure function boundary: tabular arrays and explicit parameters in, identically shaped rolled statistic arrays out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: window_size > 0
- require: values is sorted by chronological time
- ensure: output.shape == (len(values), len(agg_fns))
  Witness: monotonically increasing array; verify that the rolling mean flawlessly matches hand-calculated trailing averages for every specified window.
  Dependencies: numpy
  CDG stages covered: jane_street/market_volatility, optiver/rolling_orderbook_stats

Time-Decay Weighting

In event-driven prediction environments, the relevance and predictive power of historical interactions strictly diminish over time.[14] Determining the exact formula for this decay is a critical research question. While linear decay (where weights decrease at a constant rate until hitting zero) and logarithmic decay are occasionally utilized, exponential decay is overwhelmingly the standard in competitive machine learning due to its mathematical elegance.[24]

Exponential decay assumes the weight of an event decreases geometrically with its age, formulated continuously as $w(t) = \exp(-\lambda \Delta t)$, where λ governs the half-life of the signal.[24] The exponential formulation is favored because of its memoryless property; it allows for recursive state updates without needing to store the entire history of events, a property heavily exploited in optimizers and reinforcement learning temporal difference calculations.[26]

For pure-function tabular design, the decay function acts as a dynamic weighting mechanism

during aggregation. When computing a decayed sum, each value x_i is multiplied by $\exp(-\lambda \cdot (t_{current} - t_i))$. Because generating an $N \times N$ dense matrix of time differences for every possible pair of events is computationally unfeasible—resulting in an immediate $O(N^2)$ memory explosion—the operation must be cleverly vectorized. This involves computing relative time deltas from a fixed global epoch, scaling the values in a single 1D pass, summing them per group, and then scaling the final aggregated result back to the specific query time of the prediction.[27] This approach fundamentally restructures the algorithm to execute in $O(N)$ time.

The table below outlines the primary decay functions evaluated:

Decay Function	Formula	Algorithmic Advantage	Primary Use Case
Exponential	$w = \exp(-\lambda \Delta t)$	Supports recursive updates; never truly reaches zero.	Default risk, user engagement scoring, moving averages.
Linear	$w = \max(0, 1 - \alpha$	Hard cutoff at zero; easily interpretable.	Inventory stockouts, strict validity periods.
Logarithmic	$w = \frac{1}{\log(\Delta t + c)}$	Very slow decay for long-term memory.	Macro-economic trends, lifetime value estimation.

Name: time_decay_aggregate

Description: Compute exponentially decayed group aggregations by transforming timestamps into half-life scale factors prior to summation.

Source: https://dimacs.rutgers.edu/~graham/pubs/papers/expdecay.pdf

License: Public Domain (Mathematical formulation)

Concept type: data_assembly

Signature: (values: NDArray, timestamps: NDArray, groups: NDArray, decay_rate: float) ->

NDArray

Pure function boundary: tabular arrays and explicit parameters in, decayed summary statistics out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: decay_rate > 0
- require: len(values) == len(timestamps) == len(groups)
- ensure: output strictly bounded between min(values) and sum(values) assuming non-negative inputs
  Witness: steady stream of constant values; verify the exponentially decayed sum asymptotically approaches the theoretical geometric series limit.
  Dependencies: numpy
  CDG stages covered: home_credit/bureau_decay, h_and_m/popularity_decay

Part III: Structural Signal Transform Atoms

Transforming independent numeric and categorical features correctly maps complex data distributions into linearly separable or structurally monotonic geometries. This enables gradient boosted decision trees to efficiently partition the space with significantly fewer splits, reducing overall model depth and mitigating overfitting risks.

Feature Interactions

Decision trees fundamentally partition feature spaces using orthogonal axis-aligned splits. Consequently, they severely struggle to approximate multiplicative or ratio-based interactions implicitly. A tree requires profound depth and a massive number of nodes to accurately mimic a simple diagonal boundary or a hyperbolic curve.[29] Computing pairwise products and pairwise ratios explicitly as a preprocessing step pre-calculates these combinations, exposing the geometric relationships directly to the model.[29]

For an array of width M, generating all pairwise products yields a new feature space of $M(M-1)/2$ columns. A naive approach utilizing an outer product (numpy.outer) creates dense $N \times M \times M$ matrices. For a typical tabular dataset of $N = 100,000$ rows and $M = 1,000$ features, this results in an instantaneous 800 GB memory allocation, causing rapid Out-of-Memory (OOM) system failures.[30]

The optimal, memory-efficient pure NumPy formulation avoids computing the redundant lower triangle and the diagonal entirely. It achieves this by extracting only the upper triangle utilizing advanced boolean masking: (features[:, None, :] * features[:, :, None])[~np.tri(M, k=-1, dtype=bool)].[32] Ratios are calculated using an identical geometric masking logic but must handle division-by-zero errors by introducing a strictly positive stabilization constant ϵ to the

denominator (features + epsilon).[32] For datasets that still exceed L3 cache thresholds even after this optimization, block-wise computation via memory-mapped buffers (numpy.memmap) or iterative chunked slicing becomes an absolute necessity to preserve RAM integrity.[31]

Name: pairwise_products

Description: Compute all pairwise feature multiplications utilizing upper-triangle index masking to absolutely prevent quadratic memory allocation.

Source: https://stackoverflow.com/questions/62012339/efficiently-computing-all-pairwise-products-of-a-given-vectors-elements-in-nump

License: CC BY-SA 4.0

Concept type: signal_transform

Signature: (features: NDArray) -> NDArray

Pure function boundary: 2D numeric feature matrix in, 2D interaction matrix out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: features.ndim == 2
- ensure: output.shape == features.shape * (features.shape - 1) / 2
  Witness: small 2x3 matrix; verify the product of column 0*1, 0*2, and 1*2 flawlessly match explicitly calculated scalar counterparts.
  Dependencies: numpy
  CDG stages covered: ieee_fraud/v_feature_interactions, otto_group/product_interactions

Name: pairwise_ratios

Description: Compute pairwise feature ratios with an epsilon stabilization constant, extracting only unique variable combinations.

Source: https://stackoverflow.com/questions/68356645/how-to-efficiently-calculate-pairwise-ratios-on-rows-using-numpy

License: CC BY-SA 4.0

Concept type: signal_transform

Signature: (features: NDArray, epsilon: float) -> NDArray

Pure function boundary: 2D numeric feature matrix in, 2D interaction matrix out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: epsilon > 0
- require: features.ndim == 2
- ensure: output.shape == features.shape * (features.shape - 1) / 2
  Witness: small 2x3 matrix with zeros; verify the ratio calculations do not produce infinite values and respect the epsilon offset.
  Dependencies: numpy
  CDG stages covered: ieee_fraud/v_feature_ratios, otto_group/product_ratios

Missing Value Indicators and Imputation

Gradient boosting algorithms like XGBoost and LightGBM handle missing values internally by learning a default direction for split nodes. However, explicitly extracting boolean indicators (numpy.isnan(array).astype(int)) provides a structurally distinct feature that captures the *pattern* of missingness, which is often highly predictive of the target variable in financial and medical datasets.[34]

Furthermore, when preprocessing continuous variables through linear transformations prior to tree ingestion, NaN values must be imputed to prevent calculation cascades. A pure-function approach isolates this into two steps: a state-gathering function that computes numpy.nanmean or numpy.nanmedian over the training set, and a stateless transformation function that utilizes numpy.where(numpy.isnan(array), fill_value, array) to inject the constants.

Name: missing_indicator_and_impute

Description: Generate a binary indicator matrix for missing data and substitute NaN values with pre-calculated vectors.

Source: https://numpy.org/doc/stable/reference/generated/numpy.isnan.html

License: BSD-3-Clause

Concept type: signal_transform

Signature: (features: NDArray, fill_values: NDArray) -> tuple

Pure function boundary: 2D feature matrix with NaNs and 1D fill array in, tuple of (imputed_matrix, indicator_matrix) out; no global state or I/O.

Contracts:

- require: features.shape == len(fill_values)
- ensure: numpy.isnan(imputed_matrix).sum() == 0
  Witness: array with known NaN values; verify the resulting indicator matrix is strictly 0 and 1, and the original array contains the fill values.
  Dependencies: numpy
  CDG stages covered: porto_seguro/nan_indicators, home_credit/imputation

Rank Transformation

Mapping continuous values to their corresponding rank mitigates the impact of severe numerical outliers and structurally transforms non-stationary, heavily skewed distributions into flat, uniform distributions.[36] It is universally applied in financial domains, such as the JPX Tokyo Stock Exchange Prediction competition, to normalize highly volatile daily returns.

SciPy provides the definitive implementation via the scipy.stats.rankdata function.[37] The true algorithmic complexity of rank transformation lies entirely in its tie-breaking mechanisms.[39]

The table below outlines the distinct tie-breaking methodologies required by the contract:

Method	**Tie-Breaking Behavior**	**Result on Input ``**
average	Assigns the mean rank of the tie bracket.	[1.0, 2.5, 2.5, 4.0]
min	Assigns the lowest rank in the tie bracket.	[1.0, 2.0, 2.0, 4.0]
max	Assigns the highest rank in the tie bracket.	[1.0, 3.0, 3.0, 4.0]
dense	Ranks increase by exactly 1 regardless of tie size.	[1.0, 2.0, 2.0, 3.0]
ordinal	Enforces strict sequence order based on memory position.	[1.0, 2.0, 3.0, 4.0]

While a pure NumPy alternative devoid of SciPy dependencies can be constructed utilizing a double argsort methodology: numpy.argsort(numpy.argsort(values)) [40], this approach natively replicates only the ordinal tie-breaking strategy. It requires excessively complex boolean masking to support average or dense modes.[36] Therefore, relying on scipy.stats.rankdata provides the most mathematically sound and reliable atom.

Name: rank_transform

Description: Transform contiguous numeric vectors into fractional or dense integer ranks to enforce uniform distributions and absolute outlier robustness.

Source: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.rankdata.html

License: BSD-3-Clause

Concept type: signal_transform

Signature: (values: NDArray, method: str) -> NDArray

Pure function boundary: 1D or 2D array in, identically shaped rank array out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: method in ['average', 'min', 'max', 'dense', 'ordinal']
- ensure: output.shape == values.shape
- ensure: min(output) >= 1.0 (assuming scipy default conventions)
  Witness: array with known tied values ; verify average mode mathematically yields [1.0, 2.5, 4.0, 2.5].
  Dependencies: scipy.stats
  CDG stages covered: jpx_stock/return_ranking, porto_seguro/noise_reduction

Frequency Encoding

Frequency encoding expertly resolves high-cardinality categorical variables (e.g., zip codes, IP addresses, product hashes) by substituting the discrete textual or numerical label with its normalized occurrence count within the training dataset.[41] This aggressively compresses the dimensional space while successfully retaining popularity dynamics, effectively providing a proxy measure for structural confidence without exploding memory usage the way One-Hot Encoding does.[42]

The underlying implementation extracts unique categories and their corresponding historical frequencies via numpy.unique(categories, return_counts=True).[43] For absolute maximum execution speed, if the categorical inputs are already pre-encoded as non-negative integers, numpy.bincount directly yields the exact mapping without the $O(N \log N)$ sorting overhead of numpy.unique.

To rigorously satisfy the pure-function constraints of an atomic ML preprocessing pipeline, the "fitting" step (calculating the frequency dictionary) and the "transformation" step (applying the dictionary map to new arrays) must be structurally isolated. This is the only mathematically sound way to prevent target leakage during out-of-fold cross-validation.[41]

Name: frequency_encode

Description: Map categorical arrays to their historical frequency distributions, operating over pre-computed and isolated frequency dictionaries.

Source: https://machinelearningmastery.com/10-numpy-one-liners-to-simplify-feature-engineering/

License: MIT

Concept type: signal_transform

Signature: (categories: NDArray, frequency_map: dict) -> NDArray

Pure function boundary: categorical array and explicit mapping dictionary in, numeric frequency array out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: all values in frequency_map are >= 0
- ensure: output.shape == categories.shape
- ensure: elements in categories not present in frequency_map deterministically map to 0 or NaN
  Witness: array of discrete string elements; map via dictionary and confirm unmatched strings fall back to the mathematical baseline.
  Dependencies: numpy
  CDG stages covered: ieee_fraud/cardinality_compression, petfinder/breed_popularity

Name: frequency_encode_fit

Description: Extract the frequency distribution dictionary from a categorical array to serve as the state object for the frequency_encode atom.

Source: https://numpy.org/doc/stable/reference/generated/numpy.unique.html

License: BSD-3-Clause

Concept type: analysis

Signature: (categories: NDArray) -> dict

Pure function boundary: 1D categorical array in, dictionary mapping categories to counts out; no global state or random state.

Contracts:

- ensure: sum(output.values()) == len(categories)
  Witness: array of known repeated categories; verify output dictionary sums to the exact length of the input array.
  Dependencies: numpy
  CDG stages covered: ieee_fraud/cardinality_compression, petfinder/breed_popularity

Smoothed Target Encoding

Vanilla target encoding—the process of replacing a categorical variable with the literal mean of the target variable for that specific category—inevitably leads to catastrophic overfitting on low-frequency, highly sparse categories.[44]

Micci-Barreca's Smoothed Target Encoding permanently solves this deficiency by applying an empirical Bayesian approach.[46] The final posterior estimate assigned to the category is a convex combination of the local posterior categorical mean and the prior global mean.[45] The weighting factor $\lambda(n)$ is calculated as a logistic function based strictly on the category frequency n:

$$\lambda(n) = \frac{1}{1 + \exp(-\frac{n-k}{f})}$$

Where:

- n: The number of occurrences of the category.
- k: The inflection point (min_samples_leaf); the minimum number of samples required to trust the category average over the global average.
- f: The smoothing factor; dictates the strict steepness of the transition between the global and local mean.[47]

A pure function implementation fundamentally requires isolating the mapping generation from its application. In NumPy, this involves computing the global scalar mean, utilizing numpy.bincount to extract per-category sums and instance counts, executing the $\lambda(n)$ logistic array math, and deriving the final blended expectations in a vectorized manner.[48]

Name: target_encode

Description: Compute smoothed target encoding values for categorical features utilizing an empirical Bayesian logistic blend of global and local means.

Source: https://www.kaggle.com/code/ogrellier/python-target-encoding-for-categorical-features

License: Apache-2.0

Concept type: signal_transform

Signature: (categories: NDArray, targets: NDArray, min_samples: float, smoothing: float) -> dict

Pure function boundary: tabular arrays and explicit parameters in, encoded mapping objects out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(categories) == len(targets)
- require: min_samples >= 0 and smoothing >= 0
- ensure: every output encoding is finite and bounded by min(targets) and max(targets)
  Witness: small categorical column with sparsely repeated categories; verify smoothed means regress towards the global mean using the logistic weight formula.
  Dependencies: numpy
  CDG stages covered: home_credit/categorical_smoothing, santander/sparse_encoding

Part IV: Analytical and Gradient Boosting Atoms

Gradient boosting frameworks such as LightGBM, XGBoost, and CatBoost operate internally via highly optimized, heavily compiled C++ routines. Consequently, the actual tree-building process should be treated as an opaque atom.[49] However, configuring customized objective functions and performing programmatic feature selection requires pure Python/NumPy analytical routines acting intimately on the peripheries of the GBDT engine.

Gradient Boosting Loss Decompositions

Newton boosting evaluates the complex loss landscape by recursively calculating the first derivative (gradient) and the second derivative (hessian) of the objective function with respect to the raw prediction scores (logits).[50] Providing custom, pure-function gradient and hessian formulations empowers models to heavily optimize for highly specialized commercial objectives, such as zero-inflated insurance claims or fairness-constrained lending bounds.[53]

The following sections define the most critical custom loss formulations required for robust tabular modeling:

Tweedie Regression The Tweedie distribution is parameterized strictly by a variance power $p \in (1, 2)$ and is designed to model right-skewed, massively zero-inflated data (e.g., total insurance claims, rainfall volume).[51] Utilizing a log-link function to map logits to positive predictions, the derivatives with respect to the raw prediction score y_{pred} are computationally intensive and highly prone to numerical overflow [55]:

- **Gradient:** $\exp(y_{pred} \cdot (1-p)) - y_{true} \cdot \exp(y_{pred} \cdot (1-p))$
- **Hessian:**
  $(2-p) \cdot \exp(y_{pred} \cdot (2-p)) - (1-p) \cdot y_{true} \cdot \exp(y_{pred} \cdot (1-p))$

Name: tweedie_gradient

Description: Compute the gradient and hessian of the Tweedie deviance loss, supporting customized variance power for zero-inflated targets.

Source: https://www.kaggle.com/c/m5-forecasting-accuracy/discussion/140564

License: MIT

Concept type: analysis

Signature: (targets: NDArray, predictions: NDArray, power: float) -> tuple

Pure function boundary: arrays of identical length in, tuple of (gradient, hessian) arrays out; no global state or I/O.

Contracts:

- require: len(targets) == len(predictions)
- require: 1.0 < power < 2.0
- ensure: hessian array contains strictly positive values to allow tree splitting.
  Witness: inputs representing zero-target cases; verify gradient drives the tree correctly without math domain errors.
  Dependencies: numpy
  CDG stages covered: m5_forecasting/tweedie_custom_loss

Log-Cosh and Fair Loss The Mean Absolute Error (MAE) function is fundamentally discontinuous at exactly zero and possesses a zero-value second derivative everywhere else, mathematically stalling Newton boosting leaf-weight calculations entirely.[54] To optimize for MAE directly, Log-Cosh and Fair Loss offer strictly twice-differentiable mathematical approximations.[58]

- **Log-Cosh:** Calculates the logarithm of the hyperbolic cosine of the prediction error. It acts like MSE for small errors and MAE for large errors, perfectly blunting the impact of extreme outliers.[59]
  - Gradient: $\tanh(y_{pred} - y_{true})$
  - Hessian: $\frac{1}{\cosh^2(y_{pred} - y_{true})}$ [54]
- **Fair Loss:** Modulates quadratic behavior for small errors and absolute linear behavior for large errors, controlled dynamically via a constant parameter c.[60]
  - Gradient: $\frac{c \cdot (y_{pred} - y_{true})}{|y_{pred} - y_{true}| + c}$
  - Hessian: $\frac{c^2}{(|y_{pred} - y_{true}| + c)^2}$ [54]

Name: log_cosh_gradient

Description: Compute the first and second derivatives of the log-cosh loss function to provide a

strictly smooth, twice-differentiable approximation to Mean Absolute Error.

Source:
https://stackoverflow.com/questions/45006341/xgboost-how-to-use-mae-as-objective-function

License: CC BY-SA 4.0

Concept type: analysis

Signature: (targets: NDArray, predictions: NDArray) -> tuple

Pure function boundary: arrays of identical length in, tuple of (gradient, hessian) arrays out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(targets) == len(predictions)
- ensure: len(gradient) == len(hessian) == len(targets)
- ensure: all(hessian > 0)
  Witness: inputs representing edge-case residuals near zero; verify the gradient smoothly approaches zero without undefined discontinuities.
  Dependencies: numpy
  CDG stages covered: allstate_claims/mae_approximation, favoritia/robust_regression

Feature Importance-Based Selection (Null Importance)

Iterative feature pruning via native GBDT model interpretation mechanisms (e.g., gain or split count) is mathematically biased towards high-cardinality continuous features. The "Null Importance" paradigm corrects this structural bias by computing a baseline expected importance score under a completely non-informative, randomized setting.[62]

The algorithm requires training a baseline GBDT on the authentic target variables, extracting the feature importances, and subsequently training K separate models on randomly permuted (shuffled) versions of the target variable to build a baseline distribution of noise.[62] A pure-function atom representing this logic strictly calculates the final probability map. It evaluates the statistical likelihood that the actual importance score of a feature was randomly drawn from its respective null-distribution.[62]

To perfectly satisfy stateless requirements, the atom receives the 1D array of actual importances and the $K \times M$ matrix of null importances, returning a normalized 1D array of inclusion probabilities or log-gain ratios.

Name: null_importance_p_values

Description: Compute the statistical significance of feature importances by comparing authentic

model scores against a matrix of scores generated from randomly permuted target distributions.

Source: https://www.kaggle.com/code/ogrellier/feature-selection-with-null-importances

License: Apache-2.0

Concept type: analysis

Signature: (actual_importances: NDArray, null_importances_matrix: NDArray) -> NDArray

Pure function boundary: authentic 1D scores and $K \times M$ null score matrix in, 1D array of p-values out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: len(actual_importances) == null_importances_matrix.shape
- ensure: output values are rigidly bounded [0.0, 1.0] representing valid p-values
  Witness: matrix where actual importance strictly exceeds the absolute maximum of the null distribution; verify the resulting p-value mathematically approaches 0.
  Dependencies: numpy
  CDG stages covered: home_credit/feature_pruning, santander/null_importance

Pseudo-Labeling

In semi-supervised environments where test data features are accessible but the ultimate target labels are hidden, pseudo-labeling incorporates high-confidence test predictions back into the training corpus to improve and rigidify model decision boundaries.[65] This exact technique achieved critical success in the Mechanisms of Action (MoA) Prediction competition, where stringent classification thresholds were tightly enforced.[65]

The mathematical protocol operates systematically in multiple stages. First, an initial model is trained on X_{train}. Second, it produces a probabilistic prediction vector P_{test} on the unlabeled X_{test}. Finally, the algorithm mathematically extracts rows where the probability overwhelmingly dictates certainty, specifically where $P_{test} > \tau_{upper}$ or $P_{test} < \tau_{lower}$.[66] These specific rows are cast to rigid integers (1 or 0) and vertically concatenated directly with X_{train} for a secondary, more robust training phase.[66]

A pure-function implementation abstracts the model away entirely. It acts purely on the raw prediction arrays, accepting the probability distributions, executing boolean masking operations (e.g., predictions > upper_threshold), and returning the precise integer indices of the test records valid for semi-supervised assimilation.

Name: extract_pseudo_labels

Description: Identify high-confidence prediction indices for semi-supervised data assimilation based strictly on distinct upper and lower probability thresholds.

Source: https://www.kaggle.com/code/cdeotte/pseudo-labeling-qda-0-969

License: Apache-2.0

Concept type: analysis

Signature: (test_predictions: NDArray, upper_threshold: float, lower_threshold: float) -> tuple

Pure function boundary: probability array and threshold floats in, tuple of integer index arrays (positive_indices, negative_indices) out; no model fitting side effects, global state, random state unless explicitly passed, or file I/O.

Contracts:

- require: 0.0 <= lower_threshold < upper_threshold <= 1.0
- require: all(0.0 <= p <= 1.0 for p in test_predictions)
- ensure: len(np.intersect1d(positive_indices, negative_indices)) == 0
  Witness: probability array [0.01, 0.5, 0.99]; verify indices mapping to 0.01 and 0.99 are correctly extracted to their respective discrete bins, while 0.5 is ignored.
  Dependencies: numpy
  CDG stages covered: moa_prediction/pseudo_labeling, commonlit/semi_supervised

Conclusion

The active pursuit of hyper-optimized tabular machine learning pipelines mandates the aggressive abandonment of bloated, object-oriented data structures (such as Pandas DataFrames) in favor of low-level, pure-function array operations. As outlined exhaustively in this report, operations traditionally left to high-level libraries—such as massive group-by aggregations, relational foreign-key joins, temporal sliding windows, and mathematically complex statistical encodings—can be elegantly condensed into deterministic $O(N)$ or highly structured $O(N \log N)$ NumPy routines.

By enforcing rigid software contracts and strictly maintaining the pure-function boundaries delineated for sciona-atoms-ml, practitioners can leverage advanced hardware topologies. This ensures that granular memory management (achieved via deep stride tricks and boolean masking) and sheer execution speed (achieved via vectorized C-layer execution) are flawlessly preserved across massive multi-gigabyte datasets and mathematically complex GBDT custom objectives. The transition to atomic, stateless preprocessing represents the definitive future of high-performance tabular modeling.

Works cited

1. Implement scipy.stats.rankdata equivalent in CuPy for GPU-accelerated ranking

operations · Issue #8587 - GitHub, accessed April 29, 2026, https://github.com/cupy/cupy/issues/8587
2. Group-by From Scratch | Pythonic Perambulations, accessed April 29, 2026, https://jakevdp.github.io/blog/2017/03/22/group-by-from-scratch/
3. ml31415/numpy-groupies: Optimised tools for group-indexing operations: aggregated sum and more - GitHub, accessed April 29, 2026, https://github.com/ml31415/numpy-groupies
4. numpy-groupies · PyPI, accessed April 29, 2026, https://pypi.org/project/numpy-groupies/0.9.20/
5. numpy-groupies - PyPI, accessed April 29, 2026, https://pypi.org/project/numpy-groupies/
6. Grouping indices of unique elements in numpy - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/23268605/grouping-indices-of-unique-elements-in-numpy
7. Is there any numpy group by function? - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/38013778/is-there-any-numpy-group-by-function
8. numpy-groupies/numpy_groupies/aggregate_numba.py at master - GitHub, accessed April 29, 2026, https://github.com/ml31415/numpy-groupies/blob/master/numpy_groupies/aggregate_numba.py
9. Most efficient way to return parent rows and child aggregates in one-to-many relationship, accessed April 29, 2026, https://stackoverflow.com/questions/12219532/most-efficient-way-to-return-parent-rows-and-child-aggregates-in-one-to-many-rel
10. Child with multiple foreign keys to parent? - python - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/40929947/child-with-multiple-foreign-keys-to-parent
11. the absolute basics for beginners — NumPy v2.5.dev0 Manual, accessed April 29, 2026, https://numpy.org/devdocs/user/absolute_beginners.html
12. “PyTDL”: A versatile temporal difference learning algorithm to simulate behavior process of decision making and cognitive learning - PMC, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11743081/
13. Feature engineering for a time-series dataset - Kaggle, accessed April 29, 2026, https://www.kaggle.com/discussions/questions-and-answers/427088
14. Deep Reinforcement Learning arXiv:2201.02135v5 [cs.AI] 23 Apr 2023 - SciSpace, accessed April 29, 2026, https://scispace.com/pdf/deep-reinforcement-learning-1xbjd1kh.pdf
15. Coordinating V2V Energy Sharing for Electric Fleets via Multi-Granularity Modeling and Dynamic Spatiotemporal Matching - MDPI, accessed April 29, 2026, https://www.mdpi.com/2071-1050/17/19/8783
16. How to Vectorize the finite-difference method using NumPy array arithmetic Python?, accessed April 29, 2026,

https://stackoverflow.com/questions/56203510/how-to-vectorize-the-finite-difference-method-using-numpy-array-arithmetic-pytho
17. Efficient Rolling Statistics With NumPy - Erik Rigtorp, accessed April 29, 2026, https://rigtorp.se/2011/01/01/rolling-statistics-numpy.html
18. Rolling statistics performance: pandas vs. numpy strides - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/70368702/rolling-statistics-performance-pandas-vs-numpy-strides
19. 4.6. Using stride tricks with NumPy - IPython Cookbook, accessed April 29, 2026, https://ipython-books.github.io/46-using-stride-tricks-with-numpy/
20. numpy.lib.stride_tricks.sliding_window_view, accessed April 29, 2026, https://numpy.org/devdocs/reference/generated/numpy.lib.stride_tricks.sliding_window_view.html
21. Understanding Sliding Window in NumPy | by whyamit404 - Medium, accessed April 29, 2026, https://medium.com/@whyamit404/understanding-sliding-window-in-numpy-50c86cac822b
22. Rolling Windows in NumPy - The Backbone of Time Series Analytical Methods, accessed April 29, 2026, https://towardsdatascience.com/rolling-windows-in-numpy-the-backbone-of-time-series-analytical-methods-bc2f79ba82d2/
23. Exponentially Decayed Aggregates on Data Streams - DIMACS, accessed April 29, 2026, https://dimacs.rutgers.edu/~graham/pubs/papers/expdecay.pdf
24. Time Series - Exponential Smoothing - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/andreicosma/time-series-exponential-smoothing
25. Temporal difference learning - Wikipedia, accessed April 29, 2026, https://en.wikipedia.org/wiki/Temporal_difference_learning
26. Exponential Moving Averages at Scale: Building Smart Time-Decay Systems | by ODSC, accessed April 29, 2026, https://odsc.medium.com/exponential-moving-averages-at-scale-building-smart-time-decay-systems-45d7509cd6ae
27. Exponentially Weighted Formulation - Emergent Mind, accessed April 29, 2026, https://www.emergentmind.com/topics/exponentially-weighted-formulation
28. 10 NumPy One-Liners to Simplify Feature Engineering - MachineLearningMastery.com, accessed April 29, 2026, https://machinelearningmastery.com/10-numpy-one-liners-to-simplify-feature-engineering/
29. Optimising pairwise Euclidean distance calculations using Python - Towards Data Science, accessed April 29, 2026, https://towardsdatascience.com/optimising-pairwise-euclidean-distance-calculations-using-python-fc020112c984/
30. Solving Massive Matrix Ops with Block-Wise NumPy and Shared Memory Buffers - Medium, accessed April 29, 2026, https://medium.com/@bhagyarana80/solving-massive-matrix-ops-with-block-wise-numpy-and-shared-memory-buffers-663d1722109f

31. How to efficiently calculate pairwise ratios on rows using NumPy? - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/68356645/how-to-efficiently-calculate-pairwise-ratios-on-rows-using-numpy
32. Efficiently computing all pairwise products of a given vector's elements in NumPy, accessed April 29, 2026, https://stackoverflow.com/questions/62012339/efficiently-computing-all-pairwise-products-of-a-given-vectors-elements-in-nump
33. 1st Place - Single Model - Feature Engineering | Kaggle, accessed April 29, 2026, https://www.kaggle.com/competitions/playground-series-s5e2/discussion/565539
34. Feature Engineering: A Comprehensive Guide | Kaggle, accessed April 29, 2026, https://www.kaggle.com/discussions/general/515911
35. How to Rank Items in NumPy Array (With Examples) - Statology, accessed April 29, 2026, https://www.statology.org/numpy-rank-array/
36. rankdata — SciPy v1.17.0 Manual, accessed April 29, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mstats.rankdata.html
37. rankdata — SciPy v1.17.0 Manual, accessed April 29, 2026, https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.rankdata.html
38. How to rank Python NumPy arrays with ties? - GeeksforGeeks, accessed April 29, 2026, https://www.geeksforgeeks.org/python/how-to-rank-python-numpy-arrays-with-ties/
39. Rank items in an array using Python/NumPy, without sorting array twice - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/5284646/rank-items-in-an-array-using-python-numpy-without-sorting-array-twice
40. Machine Learning with PyTorch and Scikit-Learn, accessed April 29, 2026, https://darajat.ly/wp-content/uploads/2024/06/Machine-Learning-With-PyTorch-and-Scikit-Learn.pdf
41. Autoencoders for Anomaly Detection: Intuition to Code | Let's Data Science, accessed April 29, 2026, https://letsdatascience.com/blog/the-art-of-failing-gracefully-finding-anomalies-with-autoencoders
42. numpy fastest way to transform an array's elements to their frequency - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/32591327/numpy-fastest-way-to-transform-an-arrays-elements-to-their-frequency
43. Mapping Categorical Predictors to Numeric With Target Encoding | Blas M. Benito, PhD, accessed April 29, 2026, https://www.blasbenito.com/2023/11/15/target-encoding/
44. Target encoding done the right way - Max Halford, accessed April 29, 2026, https://maxhalford.github.io/blog/target-encoding/
45. Target Encoder: A powerful categorical encoding method - Train in Data's Blog, accessed April 29, 2026,

https://www.blog.trainindata.com/target-encoder-a-powerful-categorical-encoding-method/
46. All About Target Encoding For Classification Tasks | by Nishant Mohan - Medium, accessed April 29, 2026, https://medium.com/data-science/all-about-target-encoding-d356c4e9e82
47. Python target encoding for categorical features - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/ogrellier/python-target-encoding-for-categorical-features
48. Parameters — LightGBM 4.6.0.99 documentation, accessed April 29, 2026, https://lightgbm.readthedocs.io/en/latest/Parameters.html
49. Advanced Usage of Custom Objectives — xgboost 3.3.0-dev documentation, accessed April 29, 2026, https://xgboost.readthedocs.io/en/latest/tutorials/advanced_custom_obj.html
50. From point to probabilistic gradient boosting for claim frequency and severity prediction - PMC, accessed April 29, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12575580/
51. Building a tunable and configurable custom objective function for XGBoost - Medium, accessed April 29, 2026, https://medium.com/appsflyerengineering/building-a-tunable-and-configurable-custom-objective-function-for-xgboost-d3ced8967809
52. Zero-Inflated Tweedie Boosted Trees with CatBoost for Insurance Loss Analytics - arXiv, accessed April 29, 2026, https://arxiv.org/html/2406.16206v1
53. Xgboost-How to use "mae" as objective function? - Stack Overflow, accessed April 29, 2026, https://stackoverflow.com/questions/45006341/xgboost-how-to-use-mae-as-objective-function
54. Some Objective Function - M5 Forecasting - Accuracy | Kaggle, accessed April 29, 2026, https://www.kaggle.com/c/m5-forecasting-accuracy/discussion/140564
55. Custom Objective and Evaluation Metric — xgboost 3.3.0-dev documentation, accessed April 29, 2026, https://xgboost.readthedocs.io/en/latest/tutorials/custom_metric_obj.html
56. Gradient boosting on a loss/objective function without second derivatives - Cross Validated, accessed April 29, 2026, https://stats.stackexchange.com/questions/585613/gradient-boosting-on-a-loss-objective-function-without-second-derivatives
57. When is Log-Cosh Loss used? - Cross Validated - Stats StackExchange, accessed April 29, 2026, https://stats.stackexchange.com/questions/464354/when-is-log-cosh-loss-used
58. 5 Regression Loss Functions All Machine Learners Should Know - Comet, accessed April 29, 2026, https://www.comet.com/site/blog/5-regression-loss-functions-all-machine-learners-should-know/
59. GPBoost/include/LightGBM/config.h at master - GitHub, accessed April 29, 2026, https://github.com/fabsig/GPBoost/blob/master/include/LightGBM/config.h
60. Multivariate Forecasting of Bitcoin Volatility with Gradient Boosting: Deterministic,

Probabilistic, and Feature Importance Perspectives - arXiv, accessed April 29, 2026, https://arxiv.org/html/2511.20105v1
61. Null Importance - Medium, accessed April 29, 2026, https://medium.com/@lior_tondo/permutation-importance-e13b5cceb72c
62. Feature selection-based web application defect prediction: a systematic review, accessed April 29, 2026, https://www.emerald.com/ijwis/article/21/4/545/1275990/Feature-selection-based-web-application-defect
63. Feature Selection with Null Importances - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/ogrellier/feature-selection-with-null-importances
64. Data Science for tabular data: Advanced Techniques - Kaggle, accessed April 29, 2026, https://www.kaggle.com/code/vbmokin/data-science-for-tabular-data-advanced-techniques
65. Pseudo Labeling - QDA - [0.969], accessed April 29, 2026, https://www.kaggle.com/code/cdeotte/pseudo-labeling-qda-0-969
66. Yoonsoo Kim | Portfoly - Vercel, accessed April 29, 2026, https://portfoly-yoonniverse.vercel.app/user/62d53db4ca5fd788ffaa8727/yoonsoo-kim